Paper VIII
Second Lean Verification Block

Packet Operator Algorithms, Exact Compilation,
and GPU Realization

Komorebi P. September 6, 2026

Scope and result

This supplement continues the strengthened manuscript dated August 14, 2026. It records newly checked mathematics without silently replacing the source manuscript. The original PDF, TeX, bibliography, theorem register, source package, first supplement, and six earlier proof modules are unchanged.

Twenty-one additional manuscript items are covered. The cumulative status is **31 of 56 named manuscript-proved results**, with 25 still not fully formalized. Four classical inputs remain separately tracked. The new block adds 69 theorem declarations in five modules. Together with the earlier 60 declarations, the release contains **129 declarations across 11 modules**.

New area	Source items	Declarations
Exact compiler, family aggregation, frontier, stability, cached updates	9.4–9.7, 9.11, 9.12	21
Continuous logarithmic allocation	8.4, amended hypotheses	9
Storage, arithmetic, streaming, roofline, capacity, materialization	3.1, 3.2, 3.4, 3.6, 3.8–3.10, 4.2	22
Contracts, shape grouping, reuse thresholds, fair matching	2.9, 10.2, 10.4–10.6, 11.3	17

All 11 modules passed a fresh cumulative run. Each of the 129 theorem declarations received a separate axiom audit. Five constructed objects were audited separately and are not included in the theorem count. Both invalid controls were rejected.

The executable compiler suite passed 600 exact-rational cases against 85,320 exhaustively enumerated pruning scores. It also passed 600 cached-update comparisons against full recomputation. These finite tests are additional evidence, not replacements for the universal theorems.

Central distinction. The formalized compiler optimizes a finite, declared additive model. Neither universal structure discovery nor correctness or optimality of a CUDA backend is claimed.

1. Exact finite-tree compilation

Source items 9.4 and 9.5

A candidate tree is either a terminal node or an internal node with a nonempty finite family of children. Arbitrary finite branching is supported. Every node carries a label from which keep cost κ_v, keep error ϵ_v, and split overhead σ_v are read.

Independent admissible objects. A legal cut is defined independently of the optimizer. It either keeps the root of its subtree or, at an internal node, chooses one legal cut in every child. A terminal node can only be kept. This is the recursive form of selecting one node on each root-to-terminal path.

The cost and error of a cut L are computed recursively. With multiplier λ, its objective is

$$J_\lambda(L) = \operatorname{cost}(L) + \lambda \operatorname{err}(L).$$

Lean proves that this objective equals the recursively accumulated local keep score $\kappa_v + \lambda\epsilon_v$, with σ_v charged at every split.

Recurrence. For terminal nodes,

$$F_\lambda(v) = \kappa_v + \lambda\epsilon_v.$$

For internal nodes,

$$\boxed{F_\lambda(v) = \min\left\{\kappa_v + \lambda\epsilon_v,\ \sigma_v + \sum_{w \in \mathrm{child}(v)} F_\lambda(w)\right\}.}$$

The checked result has two parts. First, $F_\lambda(v) \le J_\lambda(L)$ for every legal cut of the subtree. Second, the constructed function `compile` returns a legal cut with objective exactly $F_\lambda(v)$. Thus the minimum is attained, not merely bounded below.

The proof proceeds by induction on the actual tree and case analysis on the independently defined cut. In the split case, it sums the child lower bounds. The returned cut chooses the lower root alternative and recursively uses attaining child cuts. Ties are resolved in favor of keeping.

Shared operator families. For reuse weights r_j, the local keep model can aggregate setup and per-operator costs:

$$\kappa_v = \kappa_v^{\mathrm{setup}} + \sum_j r_j \kappa_v^{(j)}, \qquad \epsilon_v = \sum_j r_j \epsilon_v^{(j)}.$$

Lean proves the aggregation identity at the cut level, including split overhead, before applying the compiler theorem. The proof is valid over an ordered field, including $\mathbb{Q}$ and $\mathbb{R}$. The intended nonnegative model is a specialization.

No running-time theorem is asserted for the returned-cut wrapper. It can recompute subtree scores when constructing the cut. The minimum-value recurrence and the separately checked cached-update algorithm have distinct implementations and claims.

2. Frontiers and decision stability

Source items 9.6, 9.7, and 9.11

Let L_1, L_2 be actual optima at $0 \le \lambda_1 < \lambda_2$. Their optimality inequalities are

$$\begin{aligned} \operatorname{cost}(L_1) + \lambda_1 \operatorname{err}(L_1) &\le \operatorname{cost}(L_2) + \lambda_1 \operatorname{err}(L_2), \\ \operatorname{cost}(L_2) + \lambda_2 \operatorname{err}(L_2) &\le \operatorname{cost}(L_1) + \lambda_2 \operatorname{err}(L_1). \end{aligned}$$

Adding them and using the strict multiplier ordering gives

$$\boxed{\operatorname{err}(L_2) \le \operatorname{err}(L_1), \qquad \operatorname{cost}(L_2) \ge \operatorname{cost}(L_1).}$$

These conclusions hold for any choices among tied optima. For the cuts returned by the compiler, the required optimality premises are supplied by its constructive correctness theorem.

Supported Pareto points. For $\lambda > 0$, a scalarized optimum cannot be dominated in both cost and error with one strict improvement. Lean proves this directly from the two inequalities and positivity of λ.

The converse is not asserted. For the three cost/error points

$$(0, 2), \qquad (3/2, 1), \qquad (2, 0),$$

the middle point is undominated, but it is not selected by scalarization. A checked rational inequality gives, for every $\lambda \in \mathbb{Q}$,

$$\min\{2\lambda, 2\} < 3/2 + \lambda.$$

The quantized-budget compiler, which can recover unsupported points, remains a separate unformalized item in this release.

Strict half-gap stability. At a solved internal node write

$$K = \kappa_v + \lambda \epsilon_v, \qquad S = \sigma_v + \sum_w F_\lambda(w), \qquad g = |K - S|.$$

If the updated scores satisfy

$$|K' - K| < g/2, \qquad |S' - S| < g/2,$$

Lean proves

$$K' \le S' \quad \Longleftrightarrow \quad K \le S.$$

Applying this condition recursively at every internal node proves equality of the entire returned cuts before and after perturbation, not only equality of one numerical minimum.

The hypotheses concern *solved* split scores, including changes propagated from descendants. A bound on raw local costs alone is not substituted for this condition. At an exact tie $g = 0$, the strict inequalities cannot hold. No robustness at ties is claimed.

3. Cached local recompilation

Source item 9.12

The implementation represents an edit using an explicit one-hole context. Each ancestor frame stores its parent label, the untouched sibling subtrees, the child slot containing the edit, and cached sibling optimum values. The context terminates at the root.

Let v be the new optimum of the edited subtree, and let j be the edited child slot at one frame. The update performed at that frame is

$$\boxed{\operatorname{step}(v) = \min\left\{K,\ \sigma + \sum_i \begin{cases} v, & i = j, \\ \widehat{F}_i, & i \neq j. \end{cases}\right\}.}$$

This result is passed to the next ancestor. The update function does not recursively evaluate sibling subtrees. It reads their stored values $\widehat{F}_i$.

Cache validity. A context is valid when each untouched sibling cache equals the optimum for its unchanged subtree. The function `refresh` constructs such a context from the original tree model, and Lean proves its validity. This assumption concerns old, unchanged sibling computations. It is not an assumption about the desired new root optimum.

Correctness of the actual update. For any edited subtree T' and valid context C, Lean proves

$$\operatorname{updateValue}\big(C, F(T')\big) = F\big(\operatorname{fill}(C, T')\big).$$

The proof inducts through the context. At each frame, the updated child contributes the new value, and each other child contributes its unchanged cached optimum. The frame recurrence is therefore exactly the full-tree recurrence.

The result is connected back to legal cuts: the updated value is no larger than the objective of any cut of the edited whole tree, and some cut attains it. The latter witness follows from the independently proved compiler theorem.

Unchanged context. Separate theorems show that a nonedited sibling subtree is unchanged by the child replacement, and its minimum value is consequently unchanged. A localized change of keep/error/overhead data can be represented by replacing just the corresponding subtree while leaving the surrounding model fixed.

This formalizes the dependency-local claim: recompute the edited subtree, then update the ancestor frames using old sibling caches. It does not assert that a real compiler has no nonlocal cache, rank, allocation, or device-scheduling interactions. Those interactions are outside the additive tree model.

4. The continuous allocation theorem is complete

Source item 8.4, with an explicit positive budget

Let E be a finite nonempty edge set. Assume

$$s_e > 0, \qquad c_e > 0, \qquad \tau > 0,$$

and define $C = \sum_e c_e$. The optimization problem is

$$\min_{\varepsilon_e > 0} \sum_e c_e \log \frac{1}{\varepsilon_e} \qquad \text{subject to} \qquad \sum_e s_e \varepsilon_e = \tau.$$

Lean constructs the displayed candidate

$$\boxed{\varepsilon_e^* = \frac{\tau c_e}{s_e C}}$$

and proves strict positivity and the exact budget equality. It then proves global optimality and uniqueness for the actual logarithmic objective.

Global comparison. For any other feasible ε, set $r_e = \varepsilon_e / \varepsilon_e^* > 0$. The budget identities give

$$\sum_e c_e r_e = C.$$

Using the standard analytic inequality $\log r \le r - 1$ yields

$$\sum_e c_e \log r_e \le \sum_e c_e (r_e - 1) = 0.$$

Lean separately proves the exact objective-difference identity

$$J(\varepsilon) - J(\varepsilon^*) = -\sum_e c_e \log r_e.$$

Therefore $J(\varepsilon) \ge J(\varepsilon^*)$.

If the feasible vectors differ, at least one $r_e \ne 1$. At that coordinate the logarithm inequality is strict. Since $c_e > 0$, the finite weighted sum is strict, giving

$$\varepsilon \ne \varepsilon^* \quad \Longrightarrow \quad J(\varepsilon) > J(\varepsilon^*).$$

The final theorem characterizes every feasible global minimizer as exactly the constructed formula. No unproved stationary-point, convexity, or KKT optimality assertion is supplied as a hypothesis.

Why the amendment matters. In the previous block, Lean proved that positive sensitivities and positive tolerances on a nonempty edge set force the budget to be positive. Nonpositive budgets are infeasible. This block completes the optimization theorem with that missing condition explicit. The source PDF is unchanged, and the amendment is recorded separately.

This is a real-valued mathematical optimizer. It is not an executable floating-point implementation. Clipped allocation and the discrete precision program remain outside this completed item.

5. Explicit storage and operation models

Source items 3.1, 3.2, 3.4, 3.6, 3.8–3.10, and 4.2

For a finite collection of rectangular blocks, the scalar addresses are actual elements of

$$\coprod_{e\in E} (\mathrm{Fin}(m_e) \times \mathrm{Fin}(n_e)).$$

The corresponding operation addresses include a right-hand-side index and one of two scalar-operation tags. Lean proves their cardinalities rather than merely defining a formula named "cost."

Representation	Scalar storage	Modeled work for q right sides
Dense	n^2	$2qn^2$
Packet blocks	$\sum_e m_e n_e$	$2q\sum_e m_e n_e$
Equal diagonal blocks	nb	$2qnb$
BDLR	$\sum_i n_i^2 + 2nr$	$2q(\sum_i n_i^2 + 2nr)$
Equal-block BDLR	$n(b+2r)$	$2qn(b+2r)$
Sparse plus low rank	$z + 2nr$	$2q(z+2nr)$

For equal blocks, the layout condition is $n = |I|b$. Under nonzero denominator conditions, the dense-to-block reduction is n/b and the BDLR reduction is $n/(b+2r)$. The BDLR output thin product accumulates into the diagonal result under the declared two-operations-per-coefficient model.

Streaming. The explicit event type counts one block read, one source read, and a target read plus write for every edge. With scalar size s, its byte count is

$$B_{\mathrm{stream}} = s\sum_e (m_e n_e + n_e q + 2m_e q).$$

No cross-edge cache reuse is assumed. Sparse index metadata and temporary workspace remain separate.

Capacity and roofline. Positive physical capacities and the inequalities $F \le PT$, $B \le WT$ give

$$T \ge \max\{F/P, B/W\}.$$

For positive bytes and elapsed time, the rate form is $F/T \le \min\{P, WF/B\}$. The dense memory bound is the exact equivalence $n^2 s \le M \iff n \le \sqrt{M/s}$ for nonnegative n, M and positive s.

Materialization. The declared dense product followed by one addition at each output entry has exactly $2n^2r + n^2$ modeled events and n^2 output entries. The remainder is bounded by n^2 for every n, r, giving a concrete quadratic certificate for the manuscript's $O(n^2)$ term.

These theorems count a mathematical model. They do not verify CUDA instructions, wall time, memory-system behavior, or a lower bound for every implementation.

6. Contracts, grouping, and amortization

Source items 2.9, 10.2, 10.4–10.6, and 11.3

Output conversion. A decoded run returns the actual pair

$$(\text{decode}(\text{compressed}(x)),\ t_{\text{run}}(x) + t_{\text{decode}}(\text{compressed}(x))).$$

Under the declared output equality, Lean proves that this returns the same explicit result and charges the conversion time. It also proves the downstream composition identity when compressed output is consumed directly. Omitting a positive decode cost strictly understates the modeled explicit-output time. This formalizes the protocol; it cannot certify the truth of measured times.

Shape grouping. Active shape classes are the image of the complete shape-signature map. Their count is at most the number of active blocks. The fiberwise sum over classes equals the original block sum, both for additive outputs and for modeled work. Thus grouping neither loses nor duplicates contributions. The ability to process one class in one batched launch remains a backend capability premise.

Exact profitability. With setup s, baseline time t_b, candidate time t_c, and $t_c < t_b$, Lean proves

$$\boxed{s + Rt_c < Rt_b \quad \Longleftrightarrow \quad R > \frac{s}{t_b - t_c}.}$$

For $s \geq 0$ and natural-number reuse counts, the smallest profitable count is

$$\boxed{R_{\min} = \left\lfloor \frac{s}{t_b - t_c} \right\rfloor + 1.}$$

It is proved both profitable and minimal. At equality there is only break-even. A checked example gives setup 6, baseline 5, candidate 3: three uses break even, and four are the first profitable count. Zero setup gives a first profitable count of one when there is a per-use advantage.

When the candidate is no faster per use, nonnegative setup and reuse cannot give strict profitability. Graph capture/replay is a direct specialization. For materialization, with nonnegative conversion cost and reuse, Lean proves the full equivalence: profitability requires both a lower explicit per-use time and reuse above the conversion threshold.

Fair matching. The benchmark signature has separate case, output-contract, correctness-reference, and timing-scope fields. Equality of all four fields is proved equivalent to signature equality, hence reflexive, symmetric, and transitive. Measured time is deliberately not a field in this equivalence. Matching metadata is not proof of measurement accuracy.

7. Verification and remaining work

Cumulative proof and artifact boundaries

The final run used Lean **4.34.0-rc2** with Mathlib revision

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`

All 11 modules returned exit code zero with warnings treated as errors. All 129 theorem-level audits and five construction audits were confined to the permitted standard axioms `propext`, `Classical.choice`, and `Quot.sound`, or subsets thereof. Active proof sources have no admitted proofs, new axioms, unsafe proofs, or native-decision bypasses. The false-equality and placeholder controls were both rejected.

Failures are retained. Eight of 13 captured development checks failed before correction. The source snapshots and diagnostics remain separate from the successful final run. The verifier's 17 regression tests passed. Archive recovery checked all seven supplied parts and their combined SHA-256 before successful extraction.

Executable checks. An independent exhaustive enumerator in Lean generated 85,320 pruning scores for 600 exact-rational cases. The recurrence was no larger than every enumerated score, attained an enumerated value, and agreed with the returned cut. A further 600 cached updates matched full recomputation. These tests do not add 600 theorems to the inventory.

Remaining items	Work not yet fully formalized
4.1, 4.3–4.7	Banded counts, circulant/Toeplitz, tensor contractions, Walsh recursion
5.2, 5.6, 5.8, 5.9	Rank growth, optimal recompression, product and repeated-recompression errors
6.5–6.8	Level solves, depth, inverse support, Schur support
7.2–7.4, 7.6	Krylov decomposition/bounds, block-Jacobi and PCG certificates
8.3, 8.5, 8.7	Mixed-precision chains, clipped allocation, discrete precision program
9.2, 9.8, 9.9	Block discovery certificate, budget compiler, its complexity
10.3	Optimal acyclic schedule depth

These are 25 proved manuscript items. The separate classical inputs are Eckart–Young–Mirsky, PCG convergence, the floating-point product model, and simultaneous spectral decomposition.

The original CPU numerical suite and GPU benchmarks were not rerun in this block. Papers I–VII were not rebuilt in full, and all imported Mathlib proofs were not independently rechecked. The release verifies the statements and models recorded in its ledger, not the entire paper or a production GPU implementation.

Exact commands, source and dependency hashes, axiom reports, scope ledger, tests, and failed attempts accompany the ZIP. The integrity manifest detects file changes; it is not an external authenticity signature.